

期末專題成果報告

平行化批次人臉偵測系統

B1228011 劉姮廷

1. 摘要與計畫符合度確認

本專題完成了原本規劃的正確性驗證、加速比量測與不同平行化方式比較。此外，在實驗中我更深入探討了原摘要未預期的多執行緒內鬥、行程飢餓與硬體 NUMA 架構對效能的傾斜影響，將系統迭代優化至「自適應主從架構」與「行程受控純 MPI 架構」，使其作為平行化實作與效能分析的成果。

計畫目標	執行狀態	備註
正確性驗證	完成	Serial 與平行版本皆使用灰階讀取與相同 Haar Cascade 偵測流程，High Load 皆偵測到 1,253 個人臉，Low Load 皆偵測到 1,106 個人臉
加速比與效率量測	完成	達 10.11x、92% 效率
靜態 vs 動態排程對比	完成	動態排程勝出!!
Thread Oversubscription 分析	額外完成	原計畫未涉及，實驗發現
Worker Starvation & NUMA 分析	額外完成	實作過程中觀察到的效能問題，原計畫未涉及

2. 系統架構與平行化實作演進

本專題平行化實作歷經四個階段的迭代與優化，每一階段均針對前一版本的瓶頸提出改進方案，形成完整的演進軌跡。（痛苦之歷程，我要吐了嘔嘔嘔）

2.1 序列版基準線 (Serial Baseline)

使用單核心逐張載入影像並執行 OpenCV Haar Cascade 演算法，作為效能對照的基準線。為了讓 Serial 與 Parallel 的偵測結果公平比較，我將 Serial 版本直接使用灰階模式

(IMREAD_GRAYSCALE) 讀取影像，並呼叫 `cv::setNumThreads(1)` 關閉 OpenCV 內部自動多執行緒，使其與 Controlled Pure MPI 版本採用相同的人臉偵測流程。

- 效能基準：

- High Load (248 張) 142.822 秒 / 1.74 FPS
- Low Load (600 張) 282.038 秒 / 2.13 FPS

```
(face_env) [b1228011@lgn303 face_detection_project]$ ./serial_face dataset/high_load
==== Serial Processing Start ====
Target Folder : dataset/high_load (248 images)
==== Result ====
Total Faces Detected : 1253
Total Processing Time: 142.822 seconds
Average FPS          : 1.73643
```

```
(face_env) [b1228011@lgn303 face_detection_project]$ ./serial_face dataset/low_load
==== Serial Processing Start ====
Target Folder : dataset/low_load (600 images)
==== Result ====
Total Faces Detected : 1106
Total Processing Time: 282.038 seconds
Average FPS          : 2.12738
```

2.2 純 MPI 動態排程版 (Pure MPI Dynamic Scheduling)

Master 節點 (Rank 0) 維護全域任務佇列，各 Worker 節點 (Rank > 0) 以 Pull 模式動態請求任務 (Chunk)。此架構初衷在於克服「夜間空景 (Low-Load)」與「下課尖峰 (High-Load)」影像間高達數十倍運算時間變異，達成自動負載平衡。

- Master 持續監聽 Worker 的請求訊息，收到後傳送下一批次任務索引。
- Worker 完成當前 Chunk 後立即向 Master 發送完成通知並請求新任務。
- 此 Pull Model 確保快速 Worker 自動獲得更多任務，避免 Straggler Effect。

2.3 混合平行版 (Hybrid MPI + OpenMP)

在 MPI 跨節點分派的基礎上，於每個 Worker 節點內部加入 OpenMP 多執行緒，形成雙層異質平行架構。每個 MPI 行程啟動 3 個 OpenMP 執行緒，以 4 tasks × 3 threads 配置測試。

- 第一層 (跨節點)：MPI Dynamic 排程，Chunk 粒度為 5 - 20 個影格。
- 第二層 (節點內)：OpenMP parallel for，並行處理 Chunk 內多張影格。

注意：此版本在大規模測試中因 OpenCV 隱式多執行緒與 OpenMP 執行緒庫衝突 (IPL WARN) 而產生 Thread Oversubscription，效果不如預期。

2.4 參與式與自適應主從架構 (Computing & Adaptive Master)

為解決 Master 節點在派發任務期間的 CPU 核心浪費問題，導入 Computing Master 架構，使 Rank 0 利用 MPI_Iprobe 監聽網路空檔，同時親自下場參與人臉偵測。後續針對大規模叢集引發的「行程飢餓」現象，進一步優化為 Adaptive Master：

- 總行程數 $P \leq 2$ 時：開啟參與式計算 (Computing Master 模式)。
- 總行程數 $P > 2$ 時：自動切換為專職排程者，避免 Lockstep 同步問題。

2.5 受控純 MPI 版 (Controlled Pure MPI)

透過顯式呼叫 `cv::setNumThreads(1)` 徹底封鎖 OpenCV 底層隱式多執行緒行為，將並行主導權完全交由系統級 MPI 行程控制，用以量測硬體層級最純粹、無干擾的強擴充性 (Strong Scaling) 效能。此版本最終作為本專題的強擴充性分析版本。

架構版本	核心技術	改進重點	主要發現 / 問題
Serial Baseline	OpenCV Haar	基準線	142.822s / 282.038s
Pure MPI Dynamic	MPI Pull Model	跨節點 負載均衡	np=8 效能倒退 (OpenCV 隱式 Thread)
Hybrid MPI+OpenMP	MPI + OpenMP	雙層平行	Thread 衝突，效率受限
Computing Master	MPI_Iprobe	Master CPU 利用率	大規模 Lockstep / Worker Starvation
Adaptive Master	條件式切換	自適應策略	P>2 自動轉專職調度，改善飢餓
Controlled Pure MPI ★	setNumThreads(1)	最純淨 強擴充性	10.11x 加速比， 92% 效率 ← 強擴充性分析版本

3. 實驗設計與環境配置

3.1 硬體與軟體環境

項目	規格
超算平台	國家高速網路與計算中心 (NCHC) Taiwania 3
處理器	AMD EPYC (NUMA 架構，多 NUMA 節點)
編譯器	Intel C++ Compiler (intel/2020u4)
MPI 函式庫	Intel MPI (intlempi/2020u4)
影像處理庫	OpenCV (Conda 環境 face_env，這個架設也很抽象 QAQ)
排程系統	Slurm (ctest 佇列)

3.2 實驗資料集

資料集衍生自 WIDER FACE 真實資料集，劃分為兩組極端效能壓力測試場景，以模擬現實校園監視器影像的異質性：

資料集	影像張數	偵測人臉數	場景說明
High Load (高負載)	248 張	1,253 個	人臉密集特寫，模擬下課尖峰時段
Low Load (低負載)	600 張	1,106 個	空景或單一人臉，模擬夜間校園

3.3 實驗測試矩陣

本專題共執行多組對照實驗，涵蓋以下：

- 行程數 (np)：2、4、8、12
- 架構版本：5 種 (串列 + 4 種平行)
- 資料集：High Load 與 Low Load 各自測試
- 重複次數：各配置至少執行 3 次取平均，排除系統噪音

4. 實驗結果與核心數據分析

4.1 正確性驗證

為了確認平行化後沒有造成任務遺漏或重複計算，我將 Serial Baseline 與各平行版本統一為相同的灰階讀取流程，並使用相同的 OpenCV Haar Cascade 參數進行人臉偵測。

實驗結果顯示，Serial Baseline 與各平行版本在兩組資料集上皆得到一致的偵測結果。High Load 皆偵測到 1,253 個人臉，Low Load 皆偵測到 1,106 個人臉，表示 MPI 任務切分、派發與結果彙整沒有造成影像遺漏或重複計算，結果如下表：

版本	High Load	Low Load
Serial Baseline	1,253	1,106
Pure MPI Dynamic	1,253	1,106
Hybrid MPI+OpenMP	1,253	1,106
Computing Master	1,253	1,106
Adaptive Master	1,253	1,106
Controlled Pure MPI	1,253	1,106

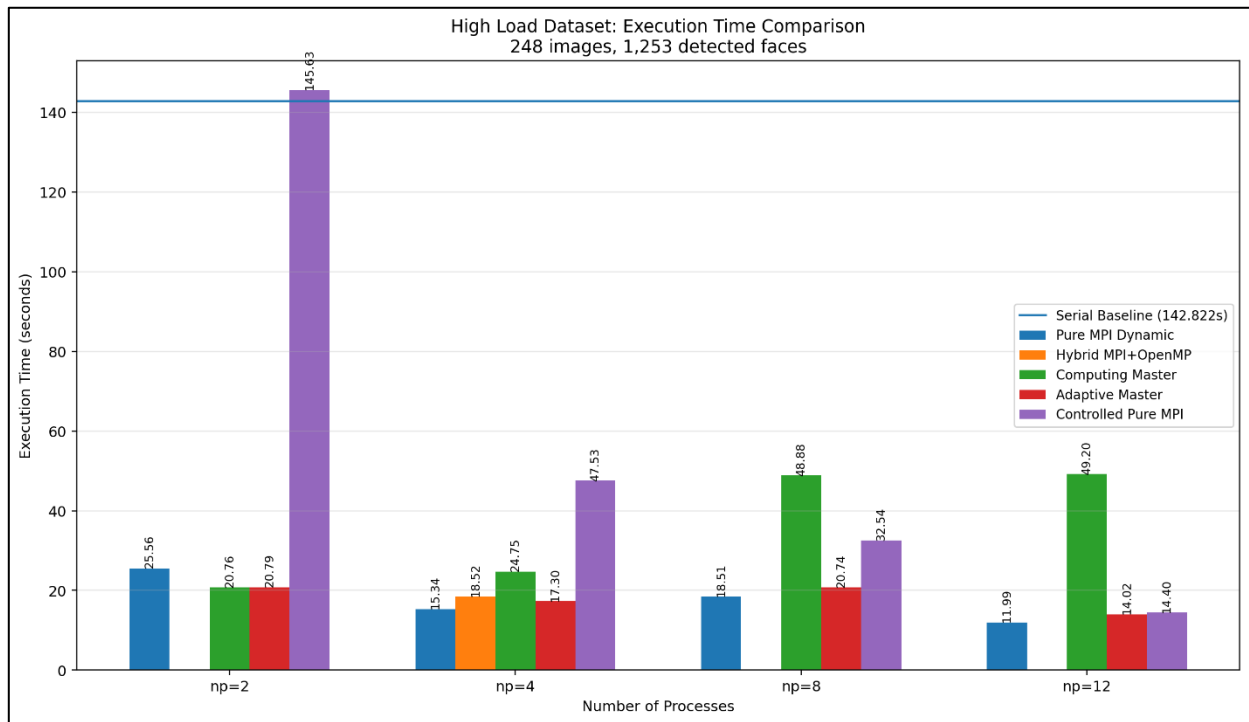
4.2 全架構執行時間總覽

架構版本	High np=2	High np=4	High np=8	High np=12	Low np=2	Low np=4	Low np=8	Low np=12
序列基準線	Serial Baseline 為 High Load 142.822 秒、Low Load 282.038 秒。 由於 Serial 版本沒有 np 參數，因此未列入 np=2/4/8/12 欄位。							
Pure MPI Dynamic	25.56	15.34	18.51	11.99	52.30	30.45	36.30	23.63
Hybrid MPI+OpenMP	—	18.52	—	—	—	37.14	—	—
Computing Master	20.76	24.75	48.88	49.20	37.56	44.18	84.92	91.45
Adaptive Master	20.79	17.30	20.74	14.02	42.64	35.01	40.70	24.80
Controlled Pure MPI ★	145.63	47.53	32.54	14.40	288.63	94.89	63.89	28.46

表 4.2 全架構版本執行時間對照（單位：秒）

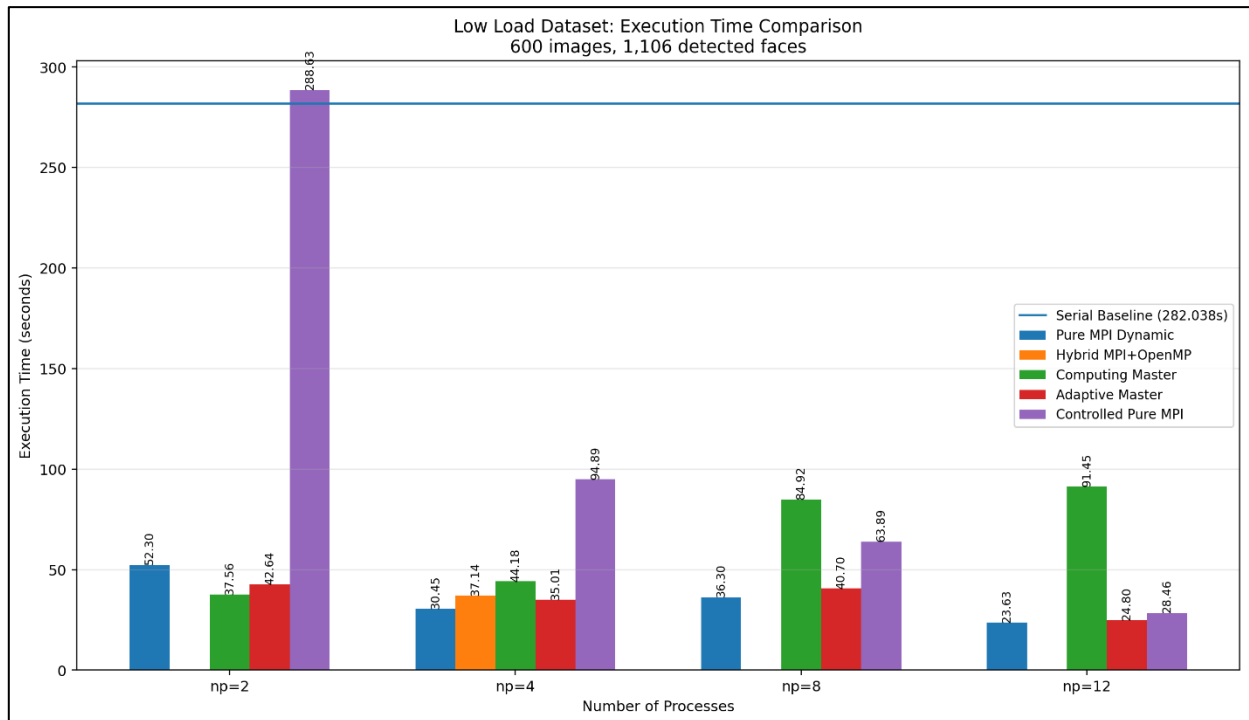
PS. 若以最短執行時間來看，Pure MPI Dynamic 在 np=12 時表現最好；但若以控制變因、避免 OpenCV 隱式多執行緒干擾來看，Controlled Pure MPI 較適合用來分析 MPI 本身的強擴充性。因此本報告將 Pure MPI Dynamic 視為實測最快版本，Controlled Pure MPI 視為較乾淨的效能分析版本。

■ High Load 執行時間比較圖



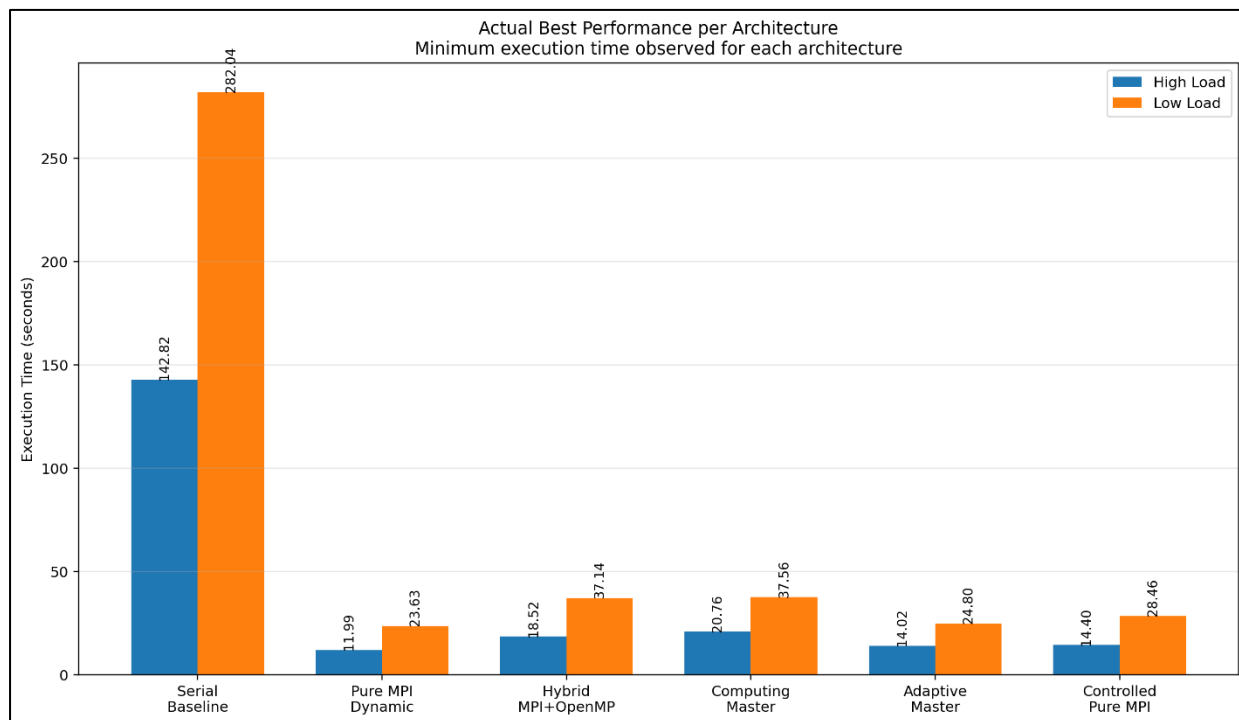
【High Load (248 張影像) 各架構執行時間比較 (np=2/4/8/12)】

■ Low Load 執行時間比較圖



【Low Load (600 張影像) 各架構執行時間比較 (np=2/4/8/12)】

■ 架構最優效能綜合比較



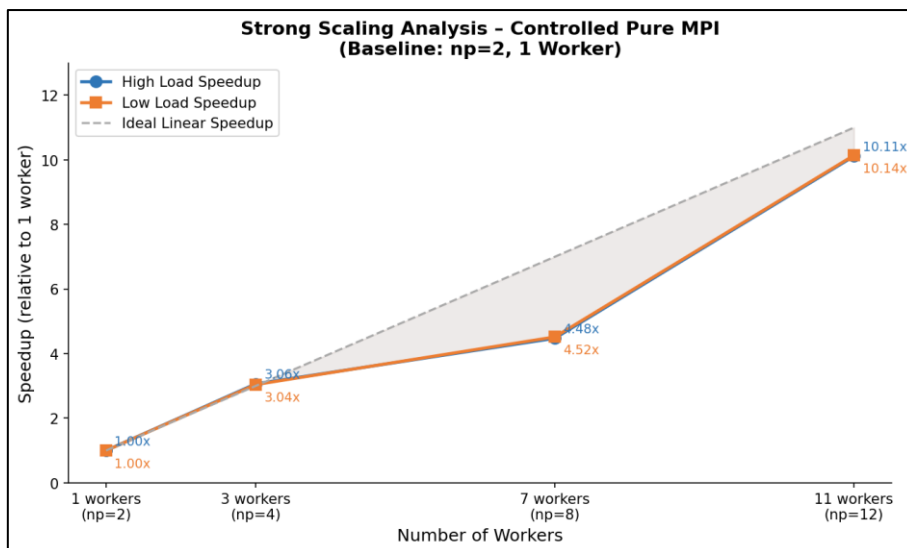
【各架構最優配置下之執行時間對比 (High/Low Load)】

4.3 強擴充性加速比分析

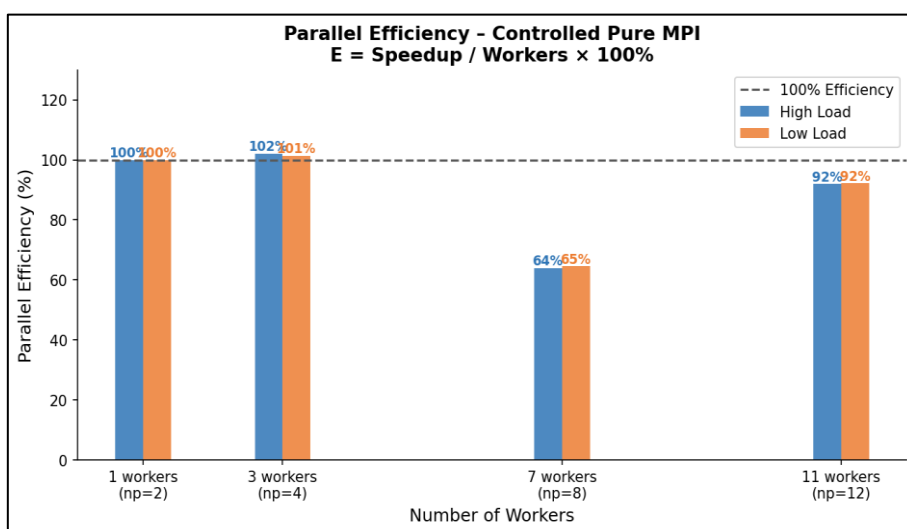
以「受控純 MPI 版」之 np=2 (1 個專職 Master + 1 個受控單執行緒 Worker) 作為最純淨的單核序列時間基準，計算不受 OpenCV 隱式執行緒干擾的強擴充性：

配置	High 時間 (s)	High 加速比	High 效率	Low 時間 (s)	Low 加速比	Low 效率	Workers
np=2 (基準)	145.63	1.00x	100%	288.63	1.00x	100%	1
np=4	47.53	3.06x	102% ★	94.89	3.04x	101%	3
np=8	32.54	4.48x	64%	63.89	4.52x	65%	7
np=12	14.40	10.11x	92% ★	28.46	10.14x	92%	11

表 4.3 強擴充性加速比與效率 (基準：np=2 Controlled Pure MPI)



【強擴充性加速比曲線 (Controlled Pure MPI)】



【各配置並行效率 (E = Speedup / Workers × 100%)】

■ 超線性加速 (Superlinear Speedup):

np=4 (3 個 Worker) 時, High Load 與 Low Load 分別達到 3.06x 與 3.04x, 並行效率高達 101%~102%。此超線性現象可能與快取效應、任務切分後的工作集變小、系統排程差異或量測噪音有關。本專題未進一步使用 perf 等工具量測 cache miss, 因此此處將其視為可能原因, 而非直接證明。

■ 極限擴充表現:

np=12 (11 個 Worker) 時, High/Low Load 加速比分別達到 10.11x 與 10.14x, 並行效率維持在 92%。此結果顯示, 在本實驗資料集與設定下, 專職 Master-Worker 架構能有效降低負載不均造成的等待時間。值得注意的是, np=8 效率僅 64% 左右, 可能與 NUMA 架構、快取競爭或系統排程有關, 仍需透過 numactl、hwloc 或 perf 等工具進一步驗證。

4.4 深度架構洞察與瓶頸分析

現象一：多執行緒內鬥 (Thread Oversubscription) 導致 np=8 效能倒退

在「原始純 MPI 動態排程」中，np=4 表現優異，但擴充至 np=8 時，時間反而出現倒退 (High Load 從 15.34 秒惡化至 18.51 秒，Low Load 從 30.45 秒惡化至 36.30 秒)。

● 根因分析：

- 由於編譯參數開啟了 `-fopenmp`，OpenCV 在執行 `detectMultiScale` 時會私自向作業系統索取多個執行緒。
- 當 MPI 行程數增至 8 時，每個行程 × OpenCV 隱式執行緒數 >> 實體核心數，導致大量執行緒競爭同一組核心。
- CPU 耗費大量開銷進行上下文切換 (Context Switching) 與 L3 快取爭奪，效能逆向衰退。

- 解決方案：使用 `cv::setNumThreads(1)` 封鎖隱式並行 (Controlled Pure MPI)，效能立即恢復正常擴充性。

現象二：Computing Master 架構中的行程飢餓 (Worker Starvation & Lockstep)

在「Computing Master」版本中，當核心數開大至 np=12 時，高負載總時間暴增至 49.20 秒，遠差於同 np 下的 Adaptive Master (14.02 秒)。

● 根因分析：

- Master (Rank 0) 下場執行 `detectMultiScale` 這一重度運算時，會長時間阻斷其外層的 `MPI_Iprobe` 任務派發迴圈。
- 實際工作量分佈顯示：Master 獨佔 83 張圖 (33.5%)，而 Worker 1 至 11 每個僅處理約 15 張 (各 6.0%)。
- 11 個 Worker 在幾毫秒內秒殺輕量影像後，集體卡在網路上排隊等待 Master 算完再派發，整個叢集陷入 Lockstep 同步鎖死，多核心優勢蕩然無存。

- 解決方案：Adaptive Master 在 $P>2$ 時自動切換為純排程者，徹底除此瓶頸。

現象三：AMD EPYC NUMA 架構與快取局部性 (Cache Locality) 效能傾斜

在「受控純 MPI 版」np=8 的工作量分佈呈現顯著非均勻性：Worker 1, 2, 5, 6, 7 各僅處理約 26-30 張影像，而 Worker 3 與 Worker 4 卻處理了高達 55 與 54 張影像 (約為其他 Worker 的兩倍)。

● 根因分析：

- 動態排程本質上是「誰跑得快誰就搶得多」。
- AMD EPYC 處理器的 NUMA 架構特性：Worker 3、4 處理較多任務，推測可能與 NUMA 架構、快取位置或系統排程有關；但本專題未使用 `numactl`、`hwloc` 或 `perf` 進一步驗證，因此此處作為效能觀察與未來分析方向。
- 因此，Worker 處理量不均的現象可視為後續進一步分析 NUMA、快取與排程策略的觀察點。

- 未來優化方向：引入 NUMA 感知親和性 (Affinity) 綁核優化，可期望在大規模節點下進一步逼近 100% 效率極限。

5. AI 工具使用說明

本專題有使用 AI 工具輔助，但主要用於語法查詢、錯誤訊息理解與報告表格整理。
程式架構、實驗流程、Slurm 測試、資料整理與結果判讀皆由本人完成。AI 工具提供的程式碼或文字內容都有經過本人修改與確認，並以實際執行結果作為最終依據。

6. 課程技術覆蓋率驗證

依課程規定，至少 50% 的方法或 API 須為課程涵蓋主題。以下逐一一列舉：

技術 / API	是否課程主題	本專題應用方式
MPI Master-Worker 動態排程	是	Pull Model 全域任務佇列，跨節點負載均衡
MPI_Iprobe 非阻塞探測	是	Computing Master 版本之非阻塞監聽機制
MPI_Send / MPI_Recv	是	任務派發與結果回傳通訊原語
OpenMP parallel for / Hybrid OpenMP test	是	Hybrid 版本中測試節點內多執行緒處理，相關結果保留於 hybrid log。
OpenMP dynamic schedule	是	Chunk 內動態排程，第二層負載均衡
Slurm 作業管理	是	超算叢集任務送出與資源分配
強擴充性 (Strong Scaling)	是	固定問題規模，量測不同 np 加速比
並行效率計算	是	$E = S/P$ ，評估資源使用率
OpenCV Haar Cascade	非課程	影像偵測演算法，非課程 API

7. 結論與未來展望

7.1 結論

本專題完成了序列版與多種平行化版本的人臉偵測程式。實驗結果顯示，對於每張影像處理時間差異大的資料集，MPI Master-Worker 動態排程比固定分配更適合，因為 Worker 完成任務後可以主動取得下一批影像，減少等待時間。

在實際執行時間上，Pure MPI Dynamic 於 np=12 時得到最佳結果；在控制 OpenCV 內部執行緒後，Controlled Pure MPI 則呈現較清楚的強擴充性趨勢。

實作過程中也發現，單純增加行程數不一定會更快，OpenCV 隱式多執行緒、Master 是否參與計算，以及硬體 NUMA 架構都可能影響效能。因此，平行化除了要把工作切開，也需要注意底層函式庫與硬體資源的互動!!

● 實作比較結果如下表：

架構版本	High Best	Low Best
Serial Baseline	142.822	282.038
Pure MPI Dynamic	11.99	23.63
Hybrid MPI+OpenMP	18.52	37.14
Adaptive Master	14.02	24.80
Controlled Pure MPI	14.40	28.46

7.2 未來展望

本專題作為概念驗證，未來可在以下方向進一步優化：

- 引入 MPI 非阻塞群體通訊 (MPI_Ialltoall) 以進一步降低通訊等待時間。
- 實作 NUMA 感知親和性 (numactl) 綁核策略，將行程固定至鄰近資料的核心，提升快取命中率。
- 結合 GPU 加速 (CUDA/OpenCL) 作為第三層加速，探索 CPU-GPU 異質平行架構。
- 擴展至即時串流 (Real-time Streaming) 場景，結合訊號驅動 (Signal-Driven) 任務觸發機制。
- 探索 Deep Learning 人臉偵測 (如 MTCNN、RetinaFace) 的 MPI 平行化，處理更高精度需求的場景。

8. 附件

- [face_detection_project](#)
- 備註:本報告中的不同架構版本為開發過程中的多次實驗結果，他們的輸出保留於 log 檔中；最終提交程式碼則以 Serial Baseline 與 Controlled Pure MPI 版本為主(因為原本的被我覆蓋掉了 QAQ)。